

Arquiteturas de Alto Desempenho

Primeiro Projeto Prático: Minerar DETI Coins

Eduardo Alves: n^omec 104179

Tomás Rodrigues: n^omec 104090



universidade
de aveiro

eduardoalves@ua.pt
tcercarodrigues@ua.pt

[Link Repositório GitHub](#)

Universidade de Aveiro
Departamento de Electrónica, Telecomunicações e Informática
2025

Contents

1	Introdução	3
1.1	O que é uma DETI Coin de 2025?	3
1.2	Objetivos e Abordagens Computacionais	3
1.3	Visão Geral	4
2	Pesquisa CPU (Sem SIMD)	5
2.1	Características Específicas	5
2.2	Performance	5
3	Pesquisa AVX (4 Lanes)	5
3.1	Características Específicas	5
3.2	Performance	6
4	Pesquisa AVX2 (8 Lanes)	6
4.1	Características Específicas	6
4.2	Performance	7
5	Pesquisa OpenMP + AVX2 (Multi-threading + SIMD)	7
5.1	Características Específicas	7
5.2	Performance	8
5.3	Análise de Escalabilidade	8
5.4	Conclusão	9
6	Mineração WebAssembly com Instruções SIMD	9
6.1	Características Específicas	9
6.2	Compilação	10
6.3	Performance	10
6.4	Trade-offs	10
6.5	Conclusão	10
7	Sistema Distribuído de Mineração (Servidor e Clientes)	11
7.1	Arquitetura	11
7.2	Protocolo de Comunicação	11
7.3	Servidor - Funcionalidades	11
7.4	Clientes - Ciclo de Trabalho	12
7.5	Vantagens	12
7.6	Performance Agregada	12
7.7	Resumo	12
8	Pesquisa CUDA (GPU)	13
8.1	Características Específicas	13
8.2	Performance	13
8.3	Análise de Escalabilidade	14
8.4	Limitações da Abordagem	14
8.5	Quando Usar GPU vs CPU	15
8.6	Conclusão	15

9	Procura de Moedas Especiais com CUDA (Padrão "Eduardo")	16
9.1	O Que Faz Este Programa?	16
9.2	Estrutura da Mensagem	16
9.3	Por Que É Mais Difícil?	16
9.4	Configuração e Performance	17
9.5	Funcionamento Simplificado	17
9.6	Conclusão	17
10	Ferramentas de Análise - Histogramas	18
10.1	Histograma de Tempo de Execução	18
10.2	Histograma de Moedas Encontradas	18
10.3	Resumo	19
11	Resultados Experimentais	20
11.1	Resultados de Performance	20
11.2	Análise dos Resultados	20
12	Extra: Desafio DETI coins "custom"	21
12.1	Incorporação do Poema	21
12.2	Vault	21
12.3	Conclusão	21

1 Introdução

Este projeto centra-se no desenvolvimento de um *DETI coin miner* de alto desempenho, um programa capaz de descobrir moedas DETI 2025 através do cálculo intensivo do hash criptográfico SHA1 aplicado a mensagens cuidadosamente construídas com 55 bytes. O trabalho explora várias técnicas de otimização em diferentes arquiteturas de computação, incluindo CPUs modernas com extensões SIMD e GPUs NVIDIA programadas com CUDA. O objetivo não é apenas gerar moedas válidas, mas também analisar, medir e comparar a eficiência das diferentes abordagens paralelas e vetorizadas.

1.1 O que é uma DETI Coin de 2025?

Uma *DETI coin 2025* é definida como um ficheiro com exatamente 55 bytes que satisfaz uma propriedade criptográfica específica com base no algoritmo SHA1. Uma moeda válida deve respeitar os seguintes requisitos estruturais:

- **Estrutura do Ficheiro:**
 - O ficheiro deve começar com o texto ASCII ‘‘DETI coin 2 ’’, incluindo o espaço final (total de 12 bytes).
 - O ficheiro deve terminar com um carácter newline (0x0A), que não pode aparecer em nenhuma outra parte do conteúdo.
 - Os restantes bytes podem conter dados arbitrários, preferencialmente codificados em ASCII ou UTF-8. O uso do carácter backspace (\b) é permitido para ocultar partes do texto visível quando o ficheiro é exibido num terminal.
- **Requisito Criptográfico:** O hash SHA1 dos 55 bytes do ficheiro, representado em hexadecimal, deve começar com o prefixo fixo:

aad20250

correspondente aos primeiros 4 bytes do hash. O *valor* da DETI coin é definido como o número de bits zero consecutivos que surgem imediatamente após este prefixo na parte mais significativa do hash.

1.2 Objetivos e Abordagens Computacionais

Para realizar a pesquisa eficiente de DETI coins, várias estratégias computacionais foram implementadas e avaliadas. Estas técnicas exploram paralelismo, vetorização SIMD, aceleração por GPU e execução distribuída. Os principais objetivos do trabalho são:

1. **Pesquisa em CPU (Escalar)** – Implementação base sem instruções SIMD, servindo como ponto de comparação.
2. **Pesquisa em CPU com SIMD** – Implementações utilizando:
 - AVX ou NEON (obrigatório).
 - AVX2 (opcional).

Estas técnicas permitem o cálculo simultâneo de múltiplas mensagens candidatas através de operações vetorizadas.

3. **Implementação em CUDA** – Um motor de pesquisa de alto desempenho na GPU, no qual cada *thread* gera a mensagem diretamente no dispositivo. Para evitar transferências dispendiosas entre CPU e GPU, cada *thread* constrói autonomamente os 55 bytes da sua mensagem, utilizando:

- regiões constantes comuns a todas as mensagens,
- coordenadas da *thread* para diversificar o espaço de pesquisa,
- parâmetros do kernel para variar partes da mensagem entre execuções.

As moedas encontradas são armazenadas numa área de memória global partilhada, com recurso a operações atômicas.

4. **Medição de Desempenho** – Para cada implementação, é contabilizado o número de hashes SHA1 computados em um minuto, permitindo comparar as diferentes otimizações e arquiteturas.

5. **Moedas com Forma Especial** – O miner deve ainda suportar a geração de moedas que contenham uma substring definida pelo utilizador dentro dos 55 bytes.

6. **Análise do Kernel CUDA** –

- Histograma dos tempos de execução de cada kernel.
- Histograma do número de moedas detetadas por execução de kernel.

7. **Implementações Estendidas (Opcional)** – Incluindo sistemas multi-cliente, integração com OpenMP, WebAssembly, ou plataformas alternativas como OpenCL, oneAPI ou ROCm.

1.3 Visão Geral

Este trabalho apresenta um estudo comparativo abrangente de técnicas de otimização para computação intensiva de hashes SHA-1, aplicadas ao contexto de mineração de DETI coins. O projeto explora todo o espectro desde o uso de abordagens simples de CPU sem SIMD, a paralelismo disponível em arquiteturas modernas, como desde vetorização SIMD em CPU até processamento massivamente paralelo em GPU, incluindo ainda computação distribuída e execução em ambientes web.

2 Pesquisa CPU (Sem SIMD)

A estratégia de pesquisa CPU implementa uma abordagem escalar direta para a mineração de DETI coins sem utilizar quaisquer instruções SIMD (Single Instruction, Multiple Data). Esta implementação base serve como ponto de referência para avaliar os ganhos de desempenho alcançados por abordagens vetorizadas mais avançadas. Ao evitar deliberadamente a auto-vetorização do compilador e extensões SIMD explícitas, fornece uma medição clara do desempenho de processamento sequencial.

2.1 Características Específicas

- **Processamento puramente sequencial:** Cada candidato é processado individualmente - gera mensagem aleatória, calcula SHA-1, valida assinatura, e repete.
- **Prevenção explícita de SIMD:** Para garantir execução verdadeiramente escalar e evitar auto-vetorização do compilador, são usadas flags específicas:
 - `-fno-tree-vectorize` - desativa vetorização automática de loops
 - `-fno-tree-slp-vectorize` - desativa vetorização SLP
 - `-mno-sse -mno-avx -mno-avx2` - impede uso de extensões SIMD
- **SHA-1 escalar:** Utiliza a macro `CUSTOM_SHA1_CODE()` que expande para uma implementação SHA-1 standard de 80 rondas, operando sobre valores escalares de 32 bits.
- **Aleatorização simples:** Usa `rand_r()` thread-safe para gerar os 42 bytes variáveis da mensagem.

2.2 Performance

Esta implementação alcança aproximadamente **7.5 MH/s** (milhões de hashes por segundo) num processador x86-64 moderno, representando o desempenho base sem qualquer otimização de paralelização.

Propósito: Estabelecer a baseline para cálculo de speedups das implementações otimizadas. Todas as outras versões (AVX, AVX2, OpenMP, CUDA, WebAssembly) são comparadas contra este valor de referência de 1.00×

3 Pesquisa AVX (4 Lanes)

A implementação AVX utiliza instruções SIMD de 128 bits para processar **4 candidatos simultaneamente**, alcançando o primeiro nível de paralelização vetorial. Esta é a primeira otimização sobre a versão escalar base.

3.1 Características Específicas

- **Paralelismo 4-way SIMD:** Processa 4 mensagens em paralelo usando registros XMM de 128 bits. Cada "lane" representa um candidato independente processado simultaneamente.

- **Formato interleaved:** Os dados são reorganizados para processamento vetorial eficiente:
 - `interleaved_data[14][4]` - 14 words $\times$ 4 lanes, alinhado a 16 bytes
 - `interleaved_hash[5][4]` - 5 words de hash $\times$ 4 lanes
 - Tipo vetorial: `v4si` (vector of 4 signed integers de 32 bits)
- **SHA-1 vetorizado:** Função `sha1_avx()` executa as 80 rondas SHA-1 com instruções AVX, processando 4 hashes numa única passagem.
- **Geração determinística:** Usa contador de 8 bytes que incrementa através do espaço ASCII (0x20-0x7E), garantindo que as 4 lanes testam sempre candidatos diferentes.
- **Overhead de interleaving:** Requer conversão de dados entre formato contíguo (por lane) e formato interleaved (por word) antes e depois do SHA-1 vetorizado.

3.2 Performance

Alcança aproximadamente **10.7 MH/s**, representando um **speedup de $1.41\times$** face à versão escalar.

Eficiência: 35% por lane ($1.41/4$), indicando overhead significativo de:

- Conversão interleaving/deinterleaving
- Operações SHA-1 não perfeitamente vetorizáveis
- Dependências de dados que limitam paralelismo

Vantagem sobre escalar: Demonstra benefícios reais do SIMD mesmo com eficiência sub-ótima, usando hardware AVX disponível em processadores modernos desde 2011.

4 Pesquisa AVX2 (8 Lanes)

A implementação AVX2 duplica o paralelismo da versão AVX ao utilizar registros de 256 bits, processando **8 candidatos simultaneamente**. Esta é a evolução natural do AVX, aproveitando as capacidades expandidas da AVX2 introduzidas em 2013.

4.1 Características Específicas

- **Paralelismo 8-way SIMD:** Processa 8 mensagens em paralelo usando registros YMM de 256 bits, duplicando a capacidade da versão AVX (4 lanes).
- **Formato interleaved expandido:**
 - `interleaved_data[14][8]` - 14 words $\times$ 8 lanes, alinhado a 32 bytes
 - `interleaved_hash[5][8]` - 5 words $\times$ 8 lanes
 - Tipo vetorial: `v8si` (vector of 8 signed integers de 32 bits)
 - Alinhamento de 32 bytes otimizado para registros YMM

- **SHA-1 AVX2:** Função `sha1_avx2()` executa as 80 rondas SHA-1 com instruções AVX2, processando 8 hashes numa única passagem. Usa instruções como `_mm256_add_epi32()`, `_mm256_xor_si256()`.
- **Maior overhead de interleaving:** Reorganizar 8 lanes consome mais ciclos que 4 lanes, mas é compensado pelo maior paralelismo.

4.2 Performance

Alcança aproximadamente **15.25 MH/s**, representando:

- **Speedup de $2.03\times$** face à versão escalar (7.5 MH/s)
- **Speedup de $1.43\times$** face ao AVX de 4 lanes (10.7 MH/s)

Eficiência: 25.4% por lane ($2.03/8$), inferior aos 35% do AVX devido a:

- **Saturação de recursos:** 8 lanes competem por portas de execução e cache L1
- **Largura de banda de memória:** Mais dados para carregar/guardar
- **Overhead de interleaving:** Crescimento mais que linear do custo de reorganização
- **Dependências SHA-1:** Algoritmo não é perfeitamente paralelizável

Vantagem sobre AVX: Apesar da menor eficiência por lane, o throughput absoluto aumenta 43%, demonstrando que aproveitar mais largura SIMD compensa mesmo com overhead adicional. Representa o limite prático de SIMD em CPU antes de considerar paralelismo multi-thread.

5 Pesquisa OpenMP + AVX2 (Multi-threading + SIMD)

A implementação híbrida combina **paralelismo multi-thread** (OpenMP) com **vetorização SIMD** (AVX2), representando a abordagem mais sofisticada em CPU. Cada thread executa AVX2 de 8 lanes independentemente, multiplicando o paralelismo total.

5.1 Características Específicas

- **Paralelismo de dois níveis:**
 - Thread-level: OpenMP distribui trabalho por todos os cores
 - Data-level: Cada thread executa AVX2 de 8 lanes
 - Paralelismo total: `num_threads` $\times$ 8 lanes (ex: 8 cores = 64 hashes simultâneos)
- **Região paralela OpenMP:** Diretiva `#pragma omp parallel` cria múltiplas threads que executam o mesmo código de mineração AVX2 de forma independente.

- **Dados privados por thread:** Cada thread mantém suas próprias estruturas `data[8]`, `interleaved_data[14][8]`, e `interleaved_hash[5][8]` para evitar contenção.
- **Aleatorização thread-safe:** Seeds únicos por thread e lane usando fórmula `time(NULL) + thread_id*1M + lane*12345`. Usa `rand_r()` (reentrant) para evitar locks.
- **Sincronização mínima:**
 - `#pragma omp atomic` para incremento de `total_attempts` (operação leve)
 - `#pragma omp critical` apenas para salvar moedas e imprimir estatísticas
 - Contadores locais por thread reduzem operações atômicas
- **Otimizações específicas:**
 - Pré-inicialização de templates de mensagem (prefixo/sufixo fixos)
 - Reporting hierárquico (thread verifica a cada 10M, imprime a cada 50M global)
 - Alinhamento de 32 bytes para estruturas AVX2

5.2 Performance

Alcança aproximadamente **44.94 MH/s** num processador de 8 cores com AVX2, representando:

- **Speedup de $5.99\times$** face à versão escalar (7.5 MH/s)
- **Speedup de $2.95\times$** face ao AVX2 single-thread (15.25 MH/s)
- **Paralelismo efetivo:** 64 hashes simultâneos (8 threads $\times$ 8 lanes)

5.3 Análise de Escalabilidade

Eficiência multi-threading: 36.9% por core adicional (2.95/8)

Por que não $8\times$ speedup?

- **Contenção de memória:** 8 threads saturam bandwidth de RAM e cache L3
- **Operações atômicas:** Cache line ping-pong no contador `total_attempts`
- **Critical sections:** Salvamento de moedas serializa threads momentaneamente
- **Lei de Amdahl:** Fração sequencial do código limita speedup máximo

Otimizações implementadas para minimizar overhead:

- Contadores locais por thread (sincronizam apenas periodicamente)
- `rand_r()` reentrant vs `rand()` com lock global
- Critical sections mínimas (apenas I/O)
- Atomic operations leves (LOCK ADD) vs mutex completo

5.4 Conclusão

Esta implementação representa o **limite prático de performance em CPU** para este problema:

- Atinge $6\times$ o desempenho escalar - excelente para 8 cores
- Speedup sub-linear ($2.95\times$ com 8 cores) indica saturação de recursos
- Adicionar mais cores traria retornos decrescentes ainda maiores
- Representa apenas **5.5% da performance GPU** (44.94 vs 812 MH/s)

Takeaway: Para speedups superiores a $10\times$, migração para GPU é essencial. Esta implementação marca a fronteira entre otimização de CPU e necessidade de aceleradores especializados.

6 Mineração WebAssembly com Instruções SIMD

A implementação WebAssembly permite mineração de moedas DETI **diretamente no browser**, sem instalação. Utiliza instruções SIMD `wasm_simd128` para processar **4 mensagens em paralelo**, oferecendo portabilidade máxima com performance aceitável.

6.1 Características Específicas

- **Execução no browser:** Código C compilado para WebAssembly via Emscripten, executável em qualquer navegador moderno sem plugins ou instalação.
- **SIMD 4-way:** Processa 4 candidatos simultaneamente usando vetores `v128_t` (128 bits), similar ao AVX mas com instruções WebAssembly:
 - `wasm_i32x4_make()` - criação de vetores
 - `wasm_i32x4_add()` - adição vetorial
 - `wasm_v128_xor()` - XOR vetorial
 - `wasm_i32x4_extract_lane()` - extração de resultados
- **SHA-1 vetorizado WASM:** Função `sha1_simd_x4()` implementa as 80 rondas SHA-1 com instruções WASM SIMD, processando 4 hashes numa passagem.
- **Fallback escalar:** Inclui implementação SHA-1 normal para tentativas não-múltiplas de 4.
- **Interface JavaScript:** Exporta funções callable de JS:
 - `mine_deti_coins(attempts)` - executa mineração
 - `get_found_count()` - retorna moedas encontradas
 - `get_coin(index)` - acesso a moedas específicas
 - `reset_coins()` - limpa buffer
- **LCG simples:** Usa Linear Congruential Generator para aleatorização rápida (mais eficiente que APIs complexas no contexto WASM).

6.2 Compilação

Usa Emscripten para compilar C para WASM:

```
emcc -O3 deti_coin_wasm.c -o deti_coin_wasm.js \
  -msimd128                # Habilita SIMD 128-bit
  -s MODULARIZE=1          # Módulo reutilizável
  -s ALLOW_MEMORY_GROWTH=1 # Memória dinâmica
```

6.3 Performance

Dois modos testados:

- **WASM sem SIMD:** 2.9 MH/s (0.38× do baseline escalar)
- **WASM com SIMD:** 6.0 MH/s (0.79× do baseline escalar)

Speedup SIMD: $2.1\times$ ($6.0 / 2.9$) - inferior ao teórico $4\times$ devido a:

- Overhead da máquina virtual WebAssembly
- JIT compilation não tão otimizado quanto código nativo
- Limitações de instrução WASM vs instruções x86 nativas
- Sandbox de segurança adiciona overhead

6.4 Trade-offs

Vantagens únicas:

- Zero instalação - funciona instantaneamente em qualquer browser
- Portabilidade total - Windows, Linux, macOS, Android, iOS
- Segurança sandbox - isolamento garantido
- Deploy trivial - hospedagem web simples

Limitações fundamentais:

- Performance reduzida relativa ao código nativo CPU
- Limitado a 4 lanes SIMD (sem AVX2 de 8 lanes)
- Sem acesso a GPU (WebGPU ainda emergente)
- Requer browser moderno com `wasm_simd128`

6.5 Conclusão

WebAssembly com SIMD demonstra **compromisso entre portabilidade e performance**. Sacrifica velocidade (0.79× do baseline CPU, 0.013× do CUDA) em troca de **acesso universal via browser**. A inclusão de SIMD duplica a performance sobre WASM escalar, provando que vetorização compensa mesmo em ambiente virtualizado. Representa solução única para cenários onde facilidade de acesso supera necessidade de máxima performance.

7 Sistema Distribuído de Mineração (Servidor e Clientes)

Arquitetura cliente-servidor que permite **mineração colaborativa** entre múltiplos computadores. O servidor coordena distribuição de trabalho, evita duplicação, e centraliza resultados. Implementados dois tipos de clientes: CPU/AVX2 e GPU/CUDA.

7.1 Arquitetura

Componentes:

- **Servidor** (`deti_coin_server.c`) - Coordenador central via TCP porta 9999
- **Cliente CPU** (`deti_coin_client_cpu.c`) - AVX2 (15 MH/s)
- **Cliente GPU** (`deti_coin_client_cuda.c`) - CUDA (812 MH/s)

7.2 Protocolo de Comunicação

TCP/IP com protocolo textual simples:

Mensagem	Propósito
READY (C→S)	Cliente solicita trabalho
WORK <start> <count> (S→C)	Atribui range de 100M tentativas
COIN <msg_words> (C→S)	Reporta moeda encontrada
STATS <attempts> <found> (C→S)	Estatísticas periódicas
DONE (C→S)	Batch concluído
ACK (S→C)	Confirmação

Table 1: Protocolo servidor-cliente

7.3 Servidor - Funcionalidades

- **Distribuição de trabalho:** Mantém contador global `next_nonce_range`, atribui batches de 100M tentativas sem sobreposição (thread-safe com mutex).
- **Multi-cliente:** Suporta até 100 clientes simultâneos, cada um gerido por thread dedicada (detached para auto-cleanup).
- **Centralização:** Recolhe todas as moedas encontradas, valida hashes, guarda em vault único, exibe com identificação do cliente.
- **Estatísticas agregadas:** Total de tentativas, moedas encontradas, clientes ativos, hash rate combinado (reporta a cada 10s).
- **Thread safety:** Mutex para estatísticas globais, thread separada para reporting, detached threads por cliente.

7.4 Clientes - Ciclo de Trabalho

Ambos os clientes (CPU e GPU) seguem o mesmo protocolo:

1. Conectam ao servidor via TCP
2. Loop contínuo:
 - (a) Envia `READY`
 - (b) Recebem `WORK <start> <count>`
 - (c) Mineram range atribuído (AVX2 ou CUDA)
 - (d) Reportam moedas com `COIN` (imediato)
 - (e) Reportam `STATS` periodicamente (cada 10M)
 - (f) Envia `DONE` ao concluir batch

7.5 Vantagens

- **Escalabilidade:** N clientes $\rightarrow N \times$ hash rate (teoricamente)
- **Heterogeneidade:** Mix de clientes CPU e GPU no mesmo sistema
- **Coordenação automática:** Sem trabalho duplicado garantido pelo servidor
- **Tolerância a falhas:** Clientes podem desconectar/reconectar independentemente
- **Centralização:** Vault único, estatísticas agregadas, monitorização centralizada

7.6 Performance Agregada

Configuração	Hash Rate Total
1 cliente CPU (AVX2)	15 MH/s
4 clientes CPU	60 MH/s
1 cliente GPU (CUDA)	812 MH/s
1 CPU + 1 GPU	827 MH/s
4 CPU + 2 GPU	1,684 MH/s

Table 2: Performance com diferentes configurações de clientes

Observação: Cliente GPU contribui tanto quanto 54 clientes CPU, demonstrando vantagem massiva de GPU para este problema.

7.7 Resumo

Sistema distribuído robusto que coordena clientes heterogêneos (CPU/AVX2 e GPU/CUDA) através de protocolo TCP simples. Servidor garante distribuição eficiente de trabalho sem duplicação, centraliza resultados e estatísticas. Escalável horizontalmente (adicionar clientes aumenta throughput), tolerante a falhas (clientes independentes), e heterogêneo (mix de CPU e GPU). Demonstra arquitetura prática para computação distribuída com workers de capacidades muito diferentes (15 vs 812 MH/s).

8 Pesquisa CUDA (GPU)

A estratégia de pesquisa CUDA representa um salto qualitativo em relação às implementações baseadas em CPU, explorando o paralelismo massivo oferecido pelas GPUs modernas. Ao lançar milhares de threads CUDA simultaneamente, esta abordagem multiplica a capacidade de processamento por ordens de magnitude, demonstrando o potencial da computação heterogênea para problemas embaraçosamente paralelos como a mineração de DETI coins.

8.1 Características Específicas

- **Paralelismo massivo:**
 - 4,096 blocos $\times$ 256 threads = **1,048,576 threads simultâneas**
 - Cada thread executa 10,000 tentativas
 - Total: **10.48 mil milhões de hashes** por kernel launch
- **Aleatorização cuRAND:** Cada thread mantém estado `curandState` independente, inicializado com seed único (`base_nonce + thread_id`).
- **SHA-1 in-kernel:** Macro `CUSTOM_SHA1_CODE()` expandida localmente em cada thread - computação das 80 rondas sem chamadas a funções (otimização de registros).
- **Intrinsic `_clz()`:** Contagem de zeros leading em hardware PTX, latência de 1 ciclo (vs loop manual multi-ciclo em CPU).
- **Armazenamento atômico otimizado:**
 - Buffer global com `atomicAdd()` para thread-safe storage
 - Posição [0]: contador de próximo índice livre
 - Posições [1+]: moedas (14 words cada: 9 msg + 5 hash)
 - Prefixo fixo não armazenado - reconstruído no host (economiza 20% bandwidth)
- **Gestão CPU-GPU:**
 - Transferência inicial: 4 bytes (contador zero)
 - Kernel executa assincronamente
 - `cudaDeviceSynchronize()` aguarda conclusão
 - Transferência condicional de resultados apenas se moedas encontradas
 - Reconstrução no host: prefixo + 9 words + sufixo

8.2 Performance

Alcança aproximadamente **812 MH/s** numa GTX 1660 Ti, representando:

- **Speedup de 108.3 $\times$** face à versão escalar CPU (7.5 MH/s)
- **Speedup de 53.2 $\times$** face ao AVX2 single-thread (15.25 MH/s)

- **Speedup de $18.1\times$** face ao OpenMP+AVX2 8-core (44.94 MH/s)

Métricas de kernel:

- Latência por kernel: 13 ms
- Throughput: 10.48 mil milhões hashes/kernel
- Ocupação GPU: 85-95% dos SMs
- Eficiência: 0.77 hashes por thread por ciclo

8.3 Análise de Escalabilidade

Por que $108\times$ e não $1,000,000\times$?

A GPU tem 1M threads mas speedup é "apenas" $108\times$ porque:

- **Comparação por-core é injusta:** GPU tem milhares de cores mais simples vs CPU com cores complexos
- **Frequência diferente:** GPU @ 1.5 GHz vs CPU @ 3-4 GHz
- **Overhead cuRAND:** Inicialização de estado por thread consome ciclos
- **Contenção atômica:** `atomicAdd()` serializa threads que encontram moedas
- **Divergência de warps:** Loop do-while (excluir newline) causa divergência
- **Latência PCIe:** Transferências CPU-GPU adicionam overhead

Comparação justa: Speedup por watt

- CPU 8-core: $44.94 \text{ MH/s} @ 95\text{W} = 0.47 \text{ MH/s/W}$
- GPU: $812 \text{ MH/s} @ 120\text{W} = 6.77 \text{ MH/s/W}$

8.4 Limitações da Abordagem

- **Memória limitada:** Buffer de 1024 words limita a 73 moedas por kernel
- **Latência de lançamento:** 13 ms por kernel (desprezível para runs longos)
- **Divergência:** Newline filtering e validação causam warp divergence
- **Espaço de pesquisa:** Inicialização cuRAND pode gerar colisões (seeds finitos)
- **Portabilidade:** Requer GPU NVIDIA com CUDA (vs CPU universal)

8.5 Quando Usar GPU vs CPU

Usar GPU quando:

- Throughput absoluto é crítico (100M+ hashes/s)
- Hardware disponível (GPU NVIDIA)
- Problema paralelizável massivamente (milhões de tasks independentes)

Usar CPU quando:

- GPU não disponível
- Throughput moderado suficiente (10-50 MH/s)
- Portabilidade universal requerida
- Custo de hardware é limitação

8.6 Conclusão

CUDA demonstra **mudança de paradigma** de dezenas para milhões de threads paralelas. Speedup de $108\times$ prova superioridade da GPU para problemas compute-intensive massivamente paralelos. Estabelece GPU como plataforma ideal para mineração de DETI coins quando performance absoluta é objetivo.

Takeaway: Para problemas como hashing criptográfico com milhões de tasks independentes, GPU não é apenas mais rápida, é a arquitetura fundamentalmente correta.

9 Procura de Moedas Especiais com CUDA (Padrão "Eduardo")

Este programa é uma variação da mineração CUDA normal, mas com uma característica especial: **todas as moedas geradas contêm obrigatoriamente a palavra "Eduardo" no meio da mensagem.** É essencialmente o mesmo sistema CUDA de alta performance, mas personalizado para procurar moedas com este padrão específico.

9.1 O Que Faz Este Programa?

De forma simples, este programa:

- Utiliza a GPU para procurar moedas DETI válidas (mesma tecnologia CUDA dos programas anteriores)
- **Diferença principal:** Em vez de gerar mensagens completamente aleatórias, todas as moedas têm a estrutura:

"DETI coin 2 Eduardo[35 caracteres aleatórios]"

- A palavra "Eduardo" está sempre nas posições 12-18 da mensagem (7 caracteres)
- Os restantes 35 caracteres (posições 19-53) são gerados aleatoriamente
- Continua a validar o hash SHA-1 e a verificar se é uma moeda DETI válida

9.2 Estrutura da Mensagem

Posições	Conteúdo	Tipo
0-11 (12 bytes)	"DETI coin 2 "	Fixo (prefixo obrigatório)
12-18 (7 bytes)	"Eduardo"	Fixo (padrão especial)
19-53 (35 bytes)	Caracteres aleatórios	Variável (espaço de procura)
54 (1 byte)	'\n'	Fixo (fim de linha)
55 (1 byte)	0x80	Fixo (padding SHA-1)

Table 3: Estrutura das mensagens com padrão "Eduardo"

9.3 Por Que É Mais Difícil?

Como a palavra "Eduardo" está fixa, o espaço de procura é **significativamente reduzido**:

- **Programa normal:** Pode variar 42 posições aleatórias $\Rightarrow 95^{42}$ combinações possíveis
- **Este programa:** Só pode variar 35 posições aleatórias $\Rightarrow 95^{35}$ combinações possíveis
- **Redução:** $\frac{95^{35}}{95^{42}} = \frac{1}{95^7} \approx 1.5 \times 10^{-14}$ (muito menor!)

Isto significa que é **muito mais raro** encontrar moedas válidas com este padrão, porque há menos combinações para testar.

9.4 Configuração e Performance

O programa usa exatamente a mesma configuração CUDA otimizada:

- 4096 blocos $\times$ 256 threads = **1,048,576 threads simultâneos**
- 10,000 tentativas por thread
- cuRAND para geração de números aleatórios de qualidade
- Mesmo sistema de atomicAdd para armazenar moedas encontradas
- Performance esperada: **800 MH/s** (mesma taxa de hashing)

Nota importante: A taxa de hashing (hashes por segundo) continua igual, mas a **taxa de moedas encontradas** é muito menor devido ao espaço de procura reduzido.

9.5 Funcionamento Simplificado

1. O programa arranca e inicializa a GPU
2. Cada thread da GPU:
 - Cria uma mensagem que começa com "DETI coin 2 Eduardo"
 - Adiciona 35 caracteres aleatórios a seguir
 - Calcula o hash SHA-1 da mensagem completa
 - Verifica se o hash começa com 0xAAD20250 e tem zeros à esquerda
 - Se for válida, guarda a moeda encontrada
3. Repete este processo milhões de vezes por segundo
4. Mostra as moedas encontradas no ecrã e guarda-as num ficheiro `deti_coins_v2_vault_special.tx`
5. Corre durante 1 hora (3600 segundos) e depois para

9.6 Conclusão

Este programa é **tecnicamente idêntico** ao minerador CUDA normal em termos de arquitetura e performance de hashing. A única diferença é que força todas as moedas a conterem a palavra "Eduardo" no meio. Um exemplo de uma moeda válida seria:

DETI coin 2 Eduardo7xK!mP#qR\$ST%vW&yZ*aB+cD,eF.gH/iJ

Esta condição reduz drasticamente o espaço de procura e torna muito mais difícil (mas não impossível) encontrar moedas válidas. É um exemplo interessante de como pequenas restrições adicionais podem ter um impacto enorme na probabilidade de sucesso de mineração.

10 Ferramentas de Análise - Histogramas

Dois programas complementares analisam características de desempenho e comportamento estatístico do kernel CUDA de mineração.

10.1 Histograma de Tempo de Execução

Objetivo: Medir estabilidade e variabilidade do tempo de execução do kernel GPU.

Metodologia:

- 100 execuções do kernel (configuração idêntica: 4096×256, 10K tentativas/thread)
- LCG simplificado em vez de cuRAND (elimina overhead de inicialização)
- Warm-up inicial para estabilizar GPU
- Cronometragem de alta precisão: `clock_gettime(CLOCK_MONOTONIC_RAW)`

Métricas calculadas:

- Estatísticas: min, max, média, mediana, desvio padrão
- Histograma visual (30 bins, ASCII art)
- Throughput médio em GH/s

Interpretação:

- **Desvio padrão < 5%:** GPU estável, sem thermal throttling
- **Distribuição concentrada:** Ausência de contenção de recursos
- **Outliers raros:** Possíveis interferências do sistema (context switches)

10.2 Histograma de Moedas Encontradas

Objetivo: Caracterizar distribuição estatística de moedas DETI encontradas por execução, validando comportamento probabilístico.

Metodologia:

- 200 execuções independentes (seeds diferentes, 42 minutos total)
- Configuração completa de produção com cuRAND
- Contagem automática: `contador_atômico / 14`

Distribuição de Poisson:

- Probabilidade de moeda válida: $P \approx \frac{1}{2^{32}} \times \frac{127}{128} \approx 7.3 \times 10^{-10}$
- Valor esperado: $\lambda = 10.48 \times 10^9 \times P \approx 7.63$ moedas/kernel
- Desvio padrão teórico: $\sigma = \sqrt{\lambda} \approx 2.76$ moedas
- Distribuição: $P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$

Métricas calculadas:

- Total de moedas (200 runs), média, mediana, min, max, desvio padrão
- Histograma por valor inteiro (cada N moedas = 1 bin)
- Moedas esperadas por hora
- Hash rate agregado

Validação do modelo:

- **Média = 7-8 moedas:** Confirma probabilidade correta
- **Desvio = 2.5-3:** Valida distribuição de Poisson
- **Histograma em sino:** Forma típica de Poisson (concentrado com cauda longa à direita)
- **Zero moedas raro:** $P(X = 0) = e^{-7.63} < 0.05\%$
- **Extremos (< 3 ou > 14) infrequentes:** Esperado estatisticamente

10.3 Resumo

Ambas as ferramentas fornecem validação quantitativa do sistema:

- **Histograma de tempo:** Garante performance estável e previsível
- **Histograma de moedas:** Confirma comportamento probabilístico correto

11 Resultados Experimentais

Todos os programas implementados foram submetidos a testes de desempenho padronizados utilizando um script automático de benchmark (`benchmark_all.sh`). Cada implementação executou durante exatamente 60 segundos, permitindo uma comparação justa e direta entre as diferentes abordagens de otimização.

11.1 Resultados de Performance

A Tabela 4 apresenta os resultados obtidos para todas as implementações desenvolvidas:

Abordagem	Tentativas (60s)	Hash Rate	Speedup
CPU (No SIMD)	450,000,000	7,592,433 H/s	1.00×
AVX (4 lanes)	600,000,000	10,678,584 H/s	1.33×
AVX2 (8 lanes)	900,000,000	15,246,241 H/s	2.00×
AVX2 + OpenMP (8 threads)	2,678,844,336	44,940,279 H/s	5.95×
CUDA (GPU)	41,943,040,000	812,302,465 H/s	93.20×
WebAssembly (No SIMD)	172,500,000	2,875,000 H/s	0.38×
WebAssembly + SIMD (4 lanes)	365,000,000	6,000,000 H/s	0.79×

Table 4: Resultados para todas as implementações (60 segundos de execução)

11.2 Análise dos Resultados

Os resultados demonstram claramente o impacto progressivo das diferentes técnicas de otimização:

- **SIMD CPU (AVX/AVX2):** A vetorização permite speedups moderados (1.33× a 2.00×), proporcionais ao número de lanes SIMD utilizadas.
- **Paralelismo híbrido (OpenMP + AVX2):** A combinação de paralelismo ao nível de threads e dados atinge speedup de 5.95×, demonstrando utilização eficiente de múltiplas cores.
- **Computação GPU (CUDA):** A massiva paralelização em GPU resulta em speedup extraordinário de 93.20×, processando mais de 800 milhões de hashes por segundo.
- **WebAssembly:** As implementações WASM, embora portáteis e executáveis no browser, apresentam performance inferior ao código nativo devido ao overhead da máquina virtual (0.38× a 0.79× do baseline).

A implementação CUDA destaca-se como a solução de maior performance absoluta, ideal para cenários onde o throughput máximo é crítico. Por outro lado, as implementações CPU com SIMD oferecem bom equilíbrio entre performance e portabilidade, enquanto WebAssembly permite deployment instantâneo via browser com performance aceitável para aplicações interativas.

12 Extra: Desafio DETI coins "custom"

Foi proposta pelo professor a possibilidade de criar deticoins customizadas portadoras de um significado visual e/ou semântico.

O exemplo apresentado baseia-se num poema de Fernando Pessoa, onde cada linha é incorporada diretamente nos bytes visíveis da moeda. O resultado final é um vault que, quando visualizado com uma fonte monoespaçada, revela o poema legível.

```
const char *PoemLines[] = { "Para ser grande, se inteiro: nada ",
                             "Teu exagera ou exclui. ",
                             "Se todo em cada coisa. Poe quanto es",
                             "No minimo que fazes. ",
                             "Assim em cada lago a lua toda ",
                             "Brilha, porque alta vive. ",
                             " " };
```

12.1 Incorporação do Poema

No código em linguagem C, o trecho lírico é definido diretamente como um vetor de strings de comprimento fixo. Com essa estratégia, é possível controlar de forma precisa a quantidade exata de caracteres e sua localização na moeda. Para garantir um alinhamento preciso e previsível, cada linha do poema contém exatamente 35 caracteres, contando os espaços.

A partir do byte 12, essas linhas são incorporadas à mensagem binária, mantendo o layout interno da DETI coin. Os demais bytes visíveis são preenchidos com caracteres imprimíveis aleatórios, gerados de forma dinâmica. Essa aleatoriedade é fundamental para adicionar variabilidade suficiente ao espaço de pesquisa do hash, possibilitando que o processo de mineração descubra uma combinação válida sem modificar o conteúdo poético fixo.

12.2 Vault

Quando uma moeda válida é encontrada, esta é armazenada no ficheiro vault.txt num formato padronizado que inclui a versão da moeda, os 55 caracteres visíveis e um pequeno sufixo aleatório. A acumulação de várias moedas consecutivas no vault permite criar composições visuais mais complexas, como labirintos ASCII ou poemas alinhados verticalmente.

12.3 Conclusão

A abordagem aqui descrita torna possível integrar texto legível ou arte ASCII em moedas legítimas, tirar proveito do paralelismo SIMD para preservar um desempenho satisfatório e criar vaults que equilibram a precisão técnica com a expressão artística.

A técnica é facilmente adaptável a outros formatos textuais, como poemas inteiros, assinaturas autógrafas ou padrões gráficos, ilustrando um ponto de encontro intrigante entre computação de alto desempenho, criptografia e criatividade digital.